

Effectiveness of a Generate→Validate→Repair Pipeline with Batfish for Intent-to-Configuration Translation: A Preregistered Paired Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a sealed paired confirmatory study (CNSM2026-GenVal-Batfish-PR-v1) to measure whether inserting a deterministic Batfish-style validator into a bounded generate→validate→repair pipeline reduces validator-detected deployment-risk configuration errors produced by an LLM intent-to-configuration translator. The design enforced shared_initial_generation (one initial model generation per task reused by both baseline and guarded conditions) to isolate repair-mediated differences from generator sampling variance. We ran 200 paired tasks (50 per prespecified stratum: IOS-like single-step, IOS-like multi-step, JunOS-like single-step, JunOS-like multi-step) with declared generator gpt-5-mini and a bounded repair budget (≤ 1 repair attempt per task). All final per-episode validator outcomes were clean in both arms (baseline = 200/200, guarded = 200/200), yielding a paired difference (guarded - baseline) = 0.0, paired-bootstrap 95% percentile CI = [0.0, 0.0] (10,000 resamples; seed = 1729), and exact McNemar two-sided $p = 1.0$. No repairs were invoked (repair_count = 0), so the preregistered secondary estimand (repair_success_rate conditional on initial errors) is not empirically computable. We explain how the preregistered shared_initial_generation contract produces this degenerate null, summarize the deterministic validator rules used to form the binary outcome, quantify the inferential consequences of zero discordant pairs, and discuss design trade-offs, construct-validity limits, and reproducible follow-ups.

I. INTRODUCTION

Automating intent-to-configuration translation with large language models (LLMs) promises to accelerate network engineering workflows but raises acute safety concerns: a single incorrect device command may cause outages, violate policy, or create routing instability. A standard mitigation pattern is generate→validate→repair: an LLM proposes candidate configuration changes, a deterministic verifier (sandbox) checks semantic and policy properties, and a repair loop attempts automated corrections or blocks unsafe outputs. This pipeline is attractive because it combines model creativity with deterministic tooling that can detect many classes of dangerous outputs before deployment.

However, quantifying a verifier’s marginal benefit requires careful design choices. Generator sampling variance can masquerade as repair benefit if the guarded condition is allowed to re-sample; conversely, eliminating generator variance reduces repair opportunity. To produce a sharply attributable confirmatory estimate of repair-mediated improvement we preregistered CNSM2026-GenVal-Batfish-PR-v1 with a paired design that enforces shared_initial_generation (one initial candidate per task reused by both arms). This contract guarantees that any

paired discordance originates from repair activity rather than generator sampling. The preregistered question therefore is conditional and causal in a specific sense: does a bounded repair loop applied to an identical initial candidate produce a different validator outcome than leaving that same candidate unchanged? We executed the sealed run and report the artifact-grounded interpretation of the observed degenerate null.

II. RELATED WORK

Recent work demonstrates rapidly evolving practice and prototypes for LLMs and agentic patterns in NetOps. Intent-to-configuration translation systems and benchmarks (e.g., Tu et al., 2024) illustrate that LLMs can synthesize vendor-like configurations from high-level intents. Agentic orchestration and self-critique frameworks propose generate→validate→repair or meta-reasoning-based correction loops to reduce harmful outputs (Wei et al., 2025; Sang, 2025). Tool-integration and observability frameworks (OctoTools, Pan et al., 2026; Praxis, Cui et al., 2026) emphasize that correctness emerges from the combined stack of models, verifiers, and traces.

Taxonomies of failure modes and red-teaming toolkits highlight adversarial and systemic risks in agentic systems and motivate layered controls including sandboxes and human-in-the-loop gates (Maharaj, 2026; PRISM, 2026). Prior studies report prototype benefits from verifier-assisted workflows but typically do not adopt preregistered paired designs that isolate repair-mediated effects from generator variance. Our experiment complements this literature by (a) preregistering a single primary estimand in a paired design, (b) enforcing shared_initial_generation to attribute any paired effect to repair, and (c) archiving deterministic per-episode validator traces and reconciliation artifacts to support reproducible interpretation.

III. METHODOLOGY

Preregistration and estimand: The study preregistration fixed the primary estimand as the paired absolute difference in per-task validator-clean rate (guarded - baseline). Each task contributes a binary paired outcome: validator-clean (no violations on the final configuration) vs validator-detected error (one or more violations). The preregistered main test is the exact McNemar two-sided test ($\alpha = 0.05$). We also compute a paired-bootstrap 95% percentile CI with 10,000 resamples (seed = 1729) as specified.

Design and sample: The confirmatory sample comprised 200 distinct intent-to-configuration tasks executed in a paired design (400 episodes total). Tasks were balanced across four prespecified strata (50 tasks each): IOS-like single-step, IOS-like multi-step, JunOS-like single-step, JunOS-like multi-step. Task generators, seeds, and inclusion rules were fixed in the preregistration and executed without post hoc exclusions. Execution completed all planned episodes with `complete_pair_count` = 200 and failed episodes = 0.

Shared_initial_generation contract and adapter behavior: To isolate repair effects we used an adapter enforcing `initial_generation_calls_per_task` = 1; each initial candidate was recorded and reused by both baseline and guarded conditions. The adapter implements a bounded generate→validate→repair protocol with `maximum_repair_calls_per_task` = 1. The declared generator family was gpt-5-mini; provider accounting confirms `model_calls_used` = 200, consistent with one generation per task under the shared contract.

Deterministic validator (harm-oracle): The primary verifier is a deterministic Batfish-style validator embedded in the harness and used to form the binary primary outcome. The verifier enforces human-readable rule classes that map to deployment risk properties: R1 reachability (required path absence), R2 ACL/policy contradictions with declared intent, R3 routing anomalies (loops, next-hop mismatches), R4 workflow/interface invariants (e.g., make-before-break), and R5 dangerous-command flags (regex-based forbidden-operation detection). Per-episode validator traces include `validation_before` and `validation_after` snapshots, `violation_code` lists, and a `violation_count` used to decide final cleanliness (`violation_count` = 0 ⇒ validator-clean).

Contamination, missingness, and auditing: Automated contamination checks were executed as preregistered and recorded no flagged items for the public release. We followed the complete-pair missingness policy; the run produced 200 complete pairs and zero technical failures. Deterministic reconciliation audited episode and provider-call accounting to confirm that shared initial generations were reused across conditions and that no inadvertent additional model calls occurred.

IV. RESULTS

Execution accounting and raw outcomes: The run completed 400 episodes (200 tasks × 2 conditions) with `model_calls_used` = 200 and no failed episodes. Aggregated scoring shows baseline validator-clean = 200/200 and guarded validator-clean = 200/200. The four-cell paired contingency counts are `n_11` = 200 (both-clean), `n_10` = 0 (guarded-only-clean), `n_01` = 0 (baseline-only-clean), `n_00` = 0 (both-error). Representative per-episode artifacts (sampled in the archived bundle) show identical `shared_initial_candidate`, `validation_before`, and `validation_after` snapshots with `violation_count` = 0 and empty violation lists.

Primary inferential results: The observed paired estimate (guarded - baseline) = 0.0. Exact McNemar two-sided p = 1.0. The paired-bootstrap percentile 95% CI (10,000 resamples;

seed = 1729) is degenerate at [0.0, 0.0]. These inferential outputs follow deterministically from the observed contingency table: zero discordant pairs (`n_10` = `n_01` = 0) force both arms to have identical empirical success rates (1.0) and therefore a zero point estimate and a degenerate bootstrap distribution.

Repair activity and secondary estimand: `validation_after.violation_count` = 0 for every shared initial candidate; therefore the bounded repair loop was never invoked (`repair_count` = 0). The preregistered secondary estimand—`repair_success_rate` conditional on initial validator-detected errors—is not empirically computable from the run and is explicitly reported as such.

Discordant-pair sensitivity: With 200 paired tasks the smallest possible non-zero absolute paired difference is $1/200 = 0.005$ (0.5 percentage points). The preregistered power calculation assumed a substantially larger discordant-pair proportion ($p_{12} + p_{21} \approx 0.25$) and targeted a 10 percentage-point absolute reduction; with zero observed discordant pairs the data are uninformative about treatment differences at those assumed rates. Deterministic reconciliation and provider-call audit confirm that the null result arises from the observed pattern of clean shared candidates rather than execution failures.

V. DISCUSSION

Artifact-grounded interpretation: The confirmatory inference is conditional: under the preregistered `shared_initial_generation` contract and the supplied synthetic 200-task corpus, the declared generator produced validator-clean shared candidates for every task and the deterministic verifier reported zero violations in every episode. With no initial failures there was nothing for a bounded repair loop to correct. Thus the empirical conclusion is that, for this generator, task bank, validator semantics, and repair budget (≤ 1), inserting the deterministic Batfish-style validator in a guarded wrapper produced no measurable paired improvement.

Why the design produced a degenerate null: `Shared_initial_generation` eliminates generator variance, which is valuable for causal attribution of repair effect but reduces the probability of observing baseline errors produced by the generator. Our preregistered objective deliberately prioritized internal validity (attribution) over sensitivity to generator stochasticity. The observed zero-discordance is therefore a predictable outcome when a relatively capable generator produces valid candidates on the supplied synthetic corpus.

Operational implications and trade-offs: Shared and independent generation address different operational questions. `Shared_initial_generation` estimates repair effectiveness conditional on a particular candidate (i.e., whether repair can improve a specific proposal). Independent generation (two model calls per task) evaluates end-to-end production risk reduction when generation is stochastic and may better align with real-world deployment where multiple samples are available. Practitioners must choose the contract matching their operational objective: attribution of repair capability vs observed

risk reduction in a deployed, sampled system. The archived execution supports reproducible follow-ups that change only contract parameters: (A) permit independent generation to increase baseline failure probability; (B) increase repair budget or enable iterative/tool-assisted repair; or (C) enrich the harm-oracle with additional vendor-specific semantics. These follow-ups are described qualitatively in the preregistration and left for future runs using the archived manifest.

Statistical and reporting notes: A degenerate bootstrap CI and McNemar $p = 1.0$ are faithful reflections of the deterministic contingency table and the preregistered inferential procedures. We explicitly report the contingency counts ($n_{11} = 200$, $n_{10} = n_{01} = n_{00} = 0$), bootstrap settings (10,000 resamples; seed = 1729), and the smallest detectable non-zero paired difference (0.5 percentage points) so readers can calibrate sensitivity.

Construct validity and limitations: The primary outcome depends critically on validator expressiveness and task-bank challenge level. Our synthetic corpus encodes common workflow constraints (make-before-break, must-be-down-for-field rules) but remains a sandbox: the Batfish-style verifier used here does not emulate every vendor nuance, telemetry source, or production operational constraint. The run used a single declared generator family (gpt-5-mini) and a single bounded repair budget (≤ 1); conclusions do not generalize across models, larger repair strategies, adversarial inputs, or more expressive harm oracles. Absence of observed repair benefit under `shared_initial_generation` does not imply absence of benefit under independent-generation or under adversarial perturbations.

Practical recommendations: For practitioners evaluating verifier-assisted pipelines, we recommend matching the experimental contract to the operational question: if one seeks to measure repair ability to alter a fixed candidate, use `shared_initial_generation`; if one seeks to measure end-to-end reduction in deployment risk under production sampling, use independent-generation with an appropriate model-call budget and repair policy. In all cases, archive per-episode validator traces and reconciliation artifacts as we did to enable exact interpretation of null or extreme outcomes.

VI. LIMITATIONS

- Confirmatory scope limited to a synthetic public task corpus and a deterministic Batfish-style sandbox; results do not generalize to production devices, multi-vendor heterogeneity, or richer harm oracles.
- Single declared model family tested (gpt-5-mini) and a single bounded repair budget (≤ 1 repair); no cross-model or multi-iteration repair claims are made.
- The preregistered `shared_initial_generation` contract produced exactly one shared initial generation per task that was reused for both conditions; this reduces sensitivity to generator stochasticity and therefore reduced the likelihood of observing discordant pairs arising purely from generation variance.

- The repair loop was never invoked because baseline produced zero validator-detected errors; therefore the preregistered secondary hypothesis about repair-success ($>50\%$) could not be empirically evaluated.
- Contamination and template-alignment risk: automated contamination checks recorded no flagged items, but pretraining memorization or template-alignment remains a construct-validity threat that automated checks cannot fully eliminate.
- Adversarial and prompt-injection scenarios were not included; the study does not assess robustness under targeted attacks.

VII. CONCLUSION

We executed a sealed preregistered paired trial to measure the marginal effect of a deterministic Batfish-style validator in a bounded generate→validate→repair pipeline under a `shared_initial_generation` contract. For the synthetic 200-task bank and declared generator (gpt-5-mini), every shared initial candidate passed the deterministic validator, yielding an exact paired difference of 0.0 (95% CI [0.0, 0.0]; McNemar $p = 1.0$). No repairs were invoked, so the preregistered secondary estimand (repair_success_rate conditional on initial error) is not estimable from these data. These artifact-grounded results do not refute the potential value of verifier-assisted workflows; rather they demonstrate that preregistered contract choices (shared vs independent generation), task-bank difficulty, validator fidelity, and repair policy jointly determine whether verifier assistance yields an observable benefit. The archived execution bundle contains deterministic per-episode traces and reconciliation artifacts sufficient to reproduce the run and to design follow-ups that trade off generator variance and repair opportunity to quantify repair-mediated gains in more adversarial or production-like settings.

DISCLOSURE STATEMENT

This study was executed autonomously after an explicit autonomy lock under the master prompt archived at `provenance/master_prompt.txt` (sha256: 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba).

Human involvement: a Research Director selected the broad topic family (Generative AI / LLMs for NetOps) and constrained the initial scope prior to the autonomy lock; all subsequent literature review, research-question selection, preregistration, code generation, experiment execution, analysis, manuscript drafting, peer-review emulation, and revision were performed autonomously by the agentic pipeline after the lock. Preregistration: CNSM2026-GenVal-Batfish-PR-v1 was sealed prior to execution. Models and runtime: the declared generator family for the archived execution was gpt-5-mini and provider-call accounting confirms the run used one model call per task under the `shared_initial_generation` contract (`model_calls_used = 200`). Orchestration and adapters: experiments used an adapter implementing a bounded generate→validate→repair transformation with at most one

repair attempt per task. Primary validator (harm oracle): a deterministic Batfish-style verifier evaluated reachability, ACL/policy compliance, routing-loop/path anomalies, interface-state/workflow invariants (e.g., make-before-break patterns), and command-safety flags; deterministic per-episode validator traces were recorded and archived. The execution contract requiring `shared_initial_generation` (one initial generation per task reused for both baseline and guarded scoring) was preregistered and executed. The submitted artifact bundle includes the full execution manifest, raw per-episode traces, and deterministic reconciliation artifacts sufficient to reproduce the run. No manual human editing of technical content, figures, tables, or references occurred after the autonomy lock.

REFERENCES

- [1] Nguyen Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313
- [2] Yunze Wei, Xiaohui Xie, Tianshuo Hu, Yiwei Zuo, Xinyi Chen, Kaiwen Chi, Yong Cui, "INTA: Intent-Based Translation for Network Configuration with LLM Agents", 2025, 10.1109/icnp65844.2025.11192391
- [3] Abdelkader Mekrache, Adlen Ksentini, "LLM-enabled Intent-driven Service Configuration for Next Generation Networks", 2024, 10.1109/netsoft60951.2024.10588881
- [4] Yinghao Sang, "AutoCrit: A Meta-Reasoning Framework for Self-Critique and Iterative Error Correction in LLM Chains-of-Thought", 2025, 10.1109/icmlca66850.2025.11336788
- [5] Lingzhe Zhang, Tong Jia, Kangjin Wang, Chiming Duan, Minghua He, Rongqian Wang, Xi Peng, Meiling Wang, Gong Zhang, Renhai Chen, Ying Li, "Towards In-Depth Root Cause Localization for Microservices with Multi-Agent Recursion-of-Thought", 2026
- [6] Shengkun Cui, Rahul Krishna, Saurabh Jha, Ravishankar K. Iyer, "Praxis: Integrating Program Analysis with Observability for Root-Cause Analysis", 2026, 10.1109/dsn69566.2026.00021
- [7] Lu Pan, Bowen Chen, Sheng Liu, Rahul Thapa, Joseph Boen, James Zou, "OctoTools: A Multi-Agent Framework with Extensible Tools for Complex Reasoning", 2026, 10.18653/v1/2026.acl-long.1
- [8] Qisheng Lu, Aoyang Fang, Junjielong Xu, Jin'ao Shang, Songhan Zhang, Yifan Yang, Xiaochuan Yan, Pinjia He, "Beyond Fault Localization: A Trajectory-Level Study of LLM Agents for Microservice Root Cause Analysis", 2026, 10.48550/arxiv.2608.21310
- [9] Sahir Maharaj, "A Taxonomy of Failure Modes in Autonomous Agentic Systems", 2026, 10.2139/ssrn.7248799